Patterns for Data-Heavy Dashboards

> Part of the React Front-End curriculum. The earlier documents cover components, hooks, routing and a typed API client. This document covers the patterns that appear once a dashboard has many pages, many independent data requests per page, state worth sharing as a link, a phone-sized audience, and an embedded conversational view: view-model builders, independent sections, URL-backed state, a page-wide shared control, mobile variants, accessibility, theming, and embedding an agent's answers.

1. Build view models in plain functions, not in components

A component that receives an API response and decides, inline, what to draw ends up with conditionals, unit conversions and "what if it's missing" logic mixed into JSX — which is hard to test and easy to get subtly wrong. Instead, put a **typed builder function** between the response and the component:

```
// lib/gasComposition.ts -- pure, no React
export function buildGasComposition(resp: GhgCompositionResponse | null, year: number):
        GasCompositionView | null {
  if (!resp) return null;
  const row = resp.years.find((y) => y.year === year);
  if (!row) return null;                                  // a missing year is omitted, not drawn as
        zero
  const gases = row.gases.filter((g) => g.value_mtco2e !== null);
  return { year, total: sum(gases), shares: gases.map((g) => ({ gas: g.gas, pct: g.share_pct })) };
}
```

Rules that keep this layer honest:

- **A missing value is omitted, never drawn as zero.** The builder returns `null` or leaves the point out; the component renders an explicit empty state. A chart that quietly plots a zero is a lie that looks like data.
- **Every figure shown comes from the API response.** Fixed copy is limited to labels and a small amount of reference text (a published range, a methodology table). Anything that is a number about the data is computed from the response, so it updates when the data does.
- **Guardrail wording lives in one module** (`climateCopy.ts`): the "this is descriptive, not a model" note, the names that must not be used for a particular quantity. Components import it; nobody retypes it in a tooltip.
- **Builders are the unit-test surface** (`gasComposition.test.ts`): fixtures with a gap in the middle, a series that ends early, an all-missing entity, and a year the response doesn't have. Component tests then only need to confirm that the builder's output is rendered.

2. Give each section its own request, so one failure doesn't blank the page

A dashboard page with six panels fed by six endpoints should not be six `await`s in one `useEffect`, where one 503 blanks everything. Use the data-fetching hook from Core Concepts, §5 **per section**:

```
function GasCompositionSection() {
  const { data, error, loading } = useAsync(() => api.getGhgComposition({ year }), [year]);
  if (loading) return <SectionSkeleton />;
  if (error)   return <SectionError message={error} />;   // this panel only
  return <GasComposition view={buildGasComposition(data, year)} />;
}
```

- A failing endpoint then leaves every other section on screen — which is exactly the right behaviour when the API is built to fail closed with a 503 that names the cause (API Design, §2.3): show that message in that panel.
- Where two sections need the same request, **share the request** (lift it, or a hook that both call) rather than firing it twice. Where one request is *optional* (a smoothing, an extra comparison), it must not gate the main content: render without it.
- When a chain of values is derived across requests (the latest year with data), pick the latest year **with data**, never "the current year" and never a row whose total is zero.
- Make the `useAsync` effect cancellable (Core Concepts, §4.1); a response for a previous parameter must not overwrite the current one.

3. Put shareable state in the URL

Anything a user would want to send to a colleague — which countries are selected, which gas, which year, which section — belongs in the URL, not only in component state:

```
const [gas, setGas] = useUrlChoice('gas', GAS_OPTIONS, 'co2');   // ?gas=methane
```

The conventions that make this safe and pleasant:

- **Validate against a fixed list.** The URL is user-editable input. `useUrlChoice` accepts a value only if it is one of the allowed options (case-insensitively) and otherwise uses the fallback, so a stale or hand-edited link can never inject an unknown value into an API call. Multi-select state (`?countries=A&countries=B`) is validated against the available pool and capped to what a chart can show.
- **The default view has a clean URL.** Setting the value to the fallback *removes* the parameter.
- **Replace, don't push,** when the user tweaks a control, so the back button doesn't have to step through every slider position; keep the hash and every other parameter.
- **Fall back to defaults, never error,** when a link names something the page can't show. This is what lets *other software* (an agent's follow-up link) build these links safely — see Agent Guardrails, §10.
- **Treat the parameter names as a public contract.** Another codebase builds links against them; add a test on each side (the agent's link test scrapes the dashboard's source for the names).

4. One page-wide control, owned in one place

A control that changes everything on a page (the year being viewed, with a Play button that animates through years) needs a single owner:

- A hook (`usePageYear`) owns the value, syncs it to `?year=`, and exposes it to every consumer — the map, the summary cards, the bars, the tables. Derived views (`yearViews.ts`) are computed in plain functions from data already fetched, so changing the year costs no extra request.
- Animation (`useYearAnimation`) is a separate hook driving the same owner; stop it on unmount, on manual change, and at the end of the range.
- Sections that stand on their own request still read the year from the owner, so the page can never show two different years under one heading.

5. Phone-sized variants: render one control, not two

Responsive layout by CSS alone breaks down when the desktop and phone controls are different components (a slider vs. a chip that opens a bottom sheet). Hiding one with CSS leaves **both in the DOM** — duplicate form controls, duplicate accessible names, duplicate tab stops, and tests that find two of everything. Choose in JavaScript instead:

```
const isPhone = useMediaQuery('(max-width: 640px)');
return isPhone ? <PageYearMobile /> : <PageYearSlider />;   // only one exists
```

Other phone-specific patterns worth knowing:

- **Sticky, one-line anchor rows** for in-page navigation, instead of a wrapping set of links. A `scroll-spy` hook marks the current section; a "current link follows clicks and deep links, not scrolling" rule avoids the highlight flickering as content moves.
- **Deep links that survive async content.** `#section` anchors land wrong when the target doesn't exist yet or the sticky header's height hasn't been measured. A `useJumpToHashOnLoad` hook waits for the target and the measured offset before scrolling.
- **Stable viewport height.** On phones the inner height *grows* when the browser toolbar collapses as you scroll, so sizing a full-height hero (or a canvas) from it resizes the picture under the user's thumb. Use a hook that follows rotations and shrinks but ignores height-only growth at an unchanged width, keeping the toolbars-showing size.
- **Honour `prefers-reduced-motion`.** Carousels start paused, and a year-by-year animation starts pinned at its last frame, for users who have asked for less movement; an explicit Play is then a request for motion.
- **Embedding a WebGL canvas** (a globe, say) with several roots on one page is a source of bugs: keep a single root for it, test it inside an iframe at phone width, and let readouts and legends span the full width on phones rather than overlaying the canvas.

6. Accessibility is a deliverable, with tokens and tests

- **Contrast is a per-theme token problem.** Define text colours as design tokens retuned for each theme and check them in *both* themes; a "weak text" colour that passes on light often fails on dark. Panels that are dark

in both themes (chart surfaces) get their own surface token, not the page's. Warning and link colours get their own theme-aware tokens too.
- **Landmarks and route announcements.** Give the page `main` / `nav` landmarks, and announce route changes to screen readers (`useRouteAnnouncements`) because a single-page app doesn't trigger the page-load announcement a normal navigation would.
- **Keyboard behaviour for custom widgets.** A bottom sheet traps focus while open and restores it on close — and re-checks the trap if focus is lost; a chart with arrow-key navigation should step across missing years rather than getting stuck.
- **Describe charts from what is drawn.** A chart's text alternative is generated from the same view model as the chart, so it can't claim a range the chart doesn't show, and notes about gaps ("the series has no value for 2040") appear in text as well as on the chart.
- **An automated audit is a starting point.** Run a structural WCAG/PWA/mobile audit, fix in stages (contrast first, then phone navigation, then the rest), and record the findings.

7. Theming with a design system, plus a user-selectable mode

If a shared design system supplies tokens and components, consume it as intended (Routing & API Integration, §4): apply one theme at the root, and never hard-code colours that the tokens already express. A user-selectable bright/dark mode is then just a switch of the theme attribute at the root, with the choice remembered in `localStorage` (guarded with try/catch since it can throw or be empty) under an **app-specific key** — several apps may share one origin, and a generic `theme` key would collide. Decide the first-visit default deliberately (a fixed brand default, or the OS preference) and write it down. Things that read a resolved colour at runtime (a chart library that needs a hex value) must re-resolve it **after the commit that flips the theme** (`useThemeColorHex`, using a layout effect): resolving during render reads the outgoing theme's value and then stays one toggle behind, which an eyeballed screenshot can miss when the two hexes look alike — compare the resolved values, not the pixels.

8. Embedding an agent's answers

When a conversational view is added to the dashboard (Conversational Agents), the frontend's job is the other half of the generative-UI contract:

- **A renderer registry keyed by tool name.** `WidgetRenderer` maps each tool's result to a real component — ideally *the same components the dashboard's own pages use*, wrapped with the tool result as props — so an answer and its page can never disagree. An unmapped tool falls back to a generic renderer rather than crashing.
- **Streaming with a one-shot client.** Native `EventSource` is GET-only, and a query needs a body, so use a fetch-based SSE client. Guard against a stale stream: tag each submit with a request id and ignore events from older ones; make `onopen` / `onerror` throw so the library's indefinite-retry default doesn't re-send a query.
- **Answer blocks are rendered, not composed.** The response carries KPIs, a source line, a badge and follow-up links/chips as separate fields; the frontend lays them out (`KpiRow`, `SourceLine`, `FollowUpLinks`,

`FollowUpChips`). It formats the numbers (the server sends the raw value, unit and decimals), which keeps typography in one place.

- **Don't print the same paragraph twice.** If a widget embeds a section that has its own explanatory note and the answer's lead already carries it, the response flags it (`lead_includes_reading_note`) so the embedded section drops its copy.
- **Keep the conversation across navigation, reset it cleanly.** Hold the thread in a provider above the router outlet so going to another page and back doesn't blank it; a `reset()` must abort the in-flight request and invalidate it, and a new answer must cancel any queued scroll restoration so the page doesn't jump to the previous answer's position.
- **Route names must not collide with proxy prefixes.** If the dev server proxies `/agent` to the backend, a page route also named `/agent` is proxied too and never renders the app. Name the page route something else (`/ask`).
- **Unlisted routes for operational pages.** An admin page lives at a URL that is deliberately absent from the navigation, gated at the edge, and excluded from the service worker's navigation fallback (see Deployment, §7).

9. Core libraries summary

Library	Used for
react-router	URL-backed state via `useLocation` / `useNavigate` (replace + keep hash)
@microsoft/fetch-event-source	POST-bodied Server-Sent Events for an agent's streamed answer
vite-plugin-pwa (Workbox)	service worker, precache, runtime caching
AG Grid / Plotly (via a design-system chart)	tabular and chart rendering
Vitest + React Testing Library	builders, hooks, page smoke tests

See also

- React Front-End curriculum index
- Core Concepts, §7 — how the design system behind §7's theming is structured and consumed
- Routing & API Integration — the typed client these sections call
- Testing — including testing builders, hooks and URL state
- Deployment — PWA caching and edge-gated routes
- Conversational Agents, Tool Calling and Guardrails — the server side of §8